

Sri Lanka Institute of Information Technology



IT23834156

V.Nirojan

Table of Contents

1. Game Selection and Environment Setup	3
1.1 Game Overview and Justification	3
1.2 Tools and Environment	3
1.3 Setup Procedure	4
2 Methodology	4
3 Static and Dynamic Analysis	4
3.1 Static Analysis	4
3.2 Dynamic Analysis	5
3.2.1 Memory Scanning for the Life Counter	5
3.2.2 Memory Write Breakpoint to Identify the Decrement Instruction	5
4 Assembly-Level Modification	6
4.1 Identifying the Exact Instruction	6
4.2 Applying NOP Patches	7
4.3 Patch Verification	7
5 Security Analysis and Discussion	8
5.1 Why This Attack is Possible in Legacy Software	8
5.1.1 No Anti-Debugging Protection	8
5.1.2 Absence of Code Obfuscation or Packing	8
5.1.3 Game State in Unprotected Memory	8
5.1.4 No Runtime Integrity Verification	8
5.2 Missing Security Mechanisms Summary	9
5.3 Mitigation Strategies for Modern Systems	9
5.3.1 Anti-Debugging Techniques	9
5.3.2 Memory Value Obfuscation	9
5.3.3 Runtime Integrity Verification	9
5.3.4 Server-Side State Validation	9
6 Conclusion	10

Reverse Engineering and Runtime Manipulation of Legacy Game Software

Game	Chicken Invaders 1 (InterAction Studios, 2002)
Executable	CI1.exe – 32-bit Windows PE, 2,849 KB
Debugger	x32dbg (TitanEngine, Aug 19 2025 build)
Platform	Windows 10 Virtual machine

1. Game Selection and Environment Setup

1.1 Game Overview and Justification

Chicken Invaders 1 is an offline, arcade-style single-player space shooter (InterAction Studios, 1999; freeware, 2002). It is a perfect target for showing reverse engineering flaws because it is compiled as an unprotected 32-bit Windows PE binary without obfuscation, anti-debugging procedures, or integrity checks. A straightforward and observable target for assembly-level alteration is the game's health/life system, which diminishes upon player damage.

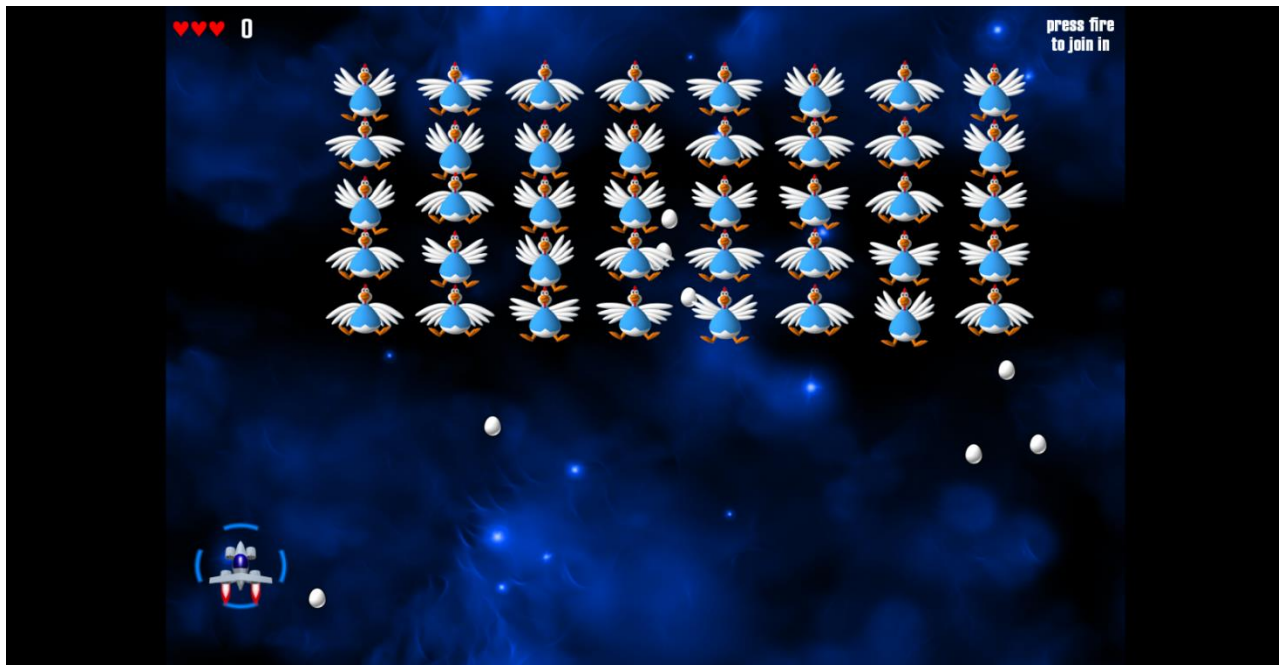


Figure 1 - Initial gameplay showing player lives before reverse engineering

1.2 Tools and Environment

- The main debugger/disassembler for static analysis, dynamic breakpoint tracing, register monitoring, assembly editing, and patch export is x32dbg (TitanEngine, August 19, 2025).
- When tracing damage events, the game's input/collision module, inputhost.dll, is recognised as the active module.
- A Windows 10 virtual machine (VM) provides a safe way to execute and instrument the game.

- Examine, confirm, and export all applied NOP patches to a new executable binary using the x32dbg Patches Dialogue.

1.3 Setup Procedure

- Installed Windows 10 (32-bit) as a virtual machine using VirtualBox to create a sandboxed environment.
- Installed x64dbg (which includes the x32dbg component) from the official GitHub repository.
- Installed Chicken Invaders v1.0 to obtain ci.exe and its associated assets.
- Disabled Windows Defender and UAC in the sandbox to prevent interference with the debugging process.
- Launched x32dbg and opened ci.exe as the target process.

2 Methodology

There were four steps in the reverse engineering process:

- In order to find a quantifiable event the decrease in player lives following damage the game executable was first started and monitored during regular gameplay.
- Second, while the player was purposefully struck, the program was loaded into the debugger and dynamically monitored. This made it possible to compare the state of the game before and after damage.
- Third, by tracking instructions that accessed or altered the observed value, the pertinent memory address and assembly instruction that reduced the life value were found.
- In order to stop the life value from declining when damage occurred, the decrement logic was finally modified at runtime by substituting a safe alternative, such NOP instructions, for the responsible instruction.

3 Static and Dynamic Analysis

3.1 Static Analysis

For static disassembly, CI1.exe was imported into x32dbg. The disassembler instantly generated a human-readable assembly listing because it was a simple 32-bit PE binary without any packing or encryption. Important findings from the static analysis:

- The binary contains no anti-disassembly, packaging, or code obfuscation techniques.
- Function boundary identification is simple because standard x86 stdcall calling procedures are used throughout.
- Player input and collision events are handled by **inputhost.dll**; functions in this module update player status, including health, in response to damage triggers.

In the CI1.exe address space, a potential health-decrement region was found at offset 0x001029AB, where a DEC instruction acts on a memory address obtained from the EDI register.

The executable was loaded into x32dbg to inspect the program structure before active debugging. Static inspection of the module showed typical executable sections such as .text, .data, and imported Windows API functions. The code section contained standard application logic with conditional jumps, arithmetic instructions, and memory write operations.

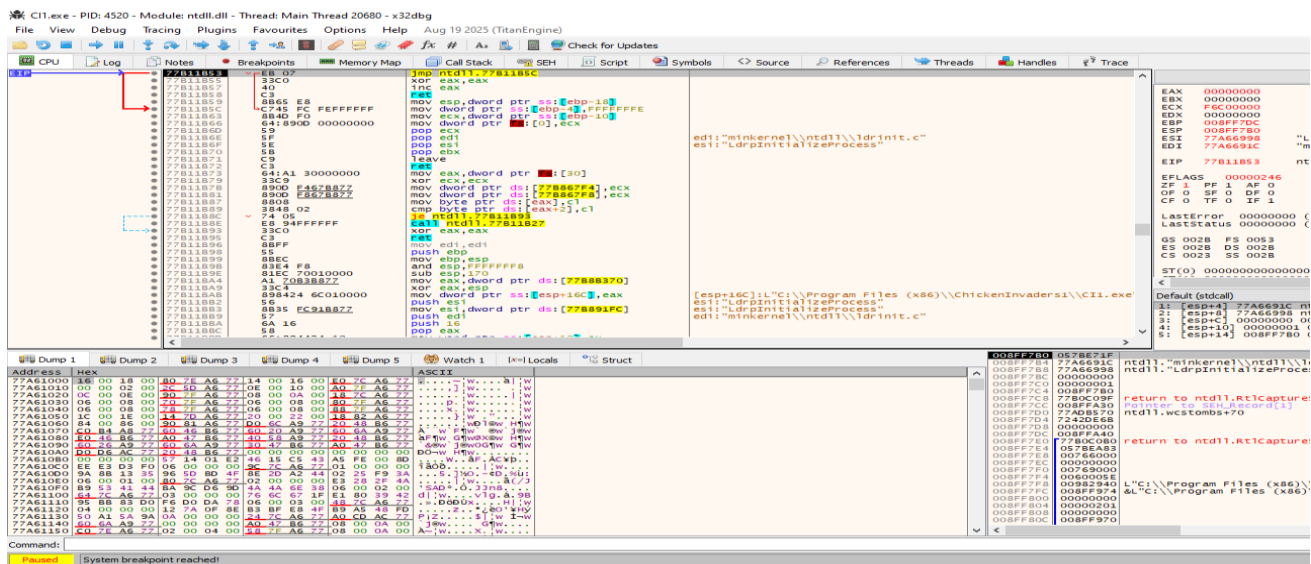


Figure 2 - x32dbg loaded with CI1.exe (PID 24372) paused at system breakpoint

3.2 Dynamic Analysis

Dynamic analysis was used to confirm the static findings and precisely identify the runtime memory location of the life counter.

3.2.1 Memory Scanning for the Life Counter

At the entry point, x32dbg was connected and the game was started. The built-in memory scanner in x32dbg was utilized in the following iterative scan procedure (no external tool):

- Three lives were noted as the initial life count. Initial Scan: Type = 4-byte integer, Value = 3 (exact). Several possible addresses from the heap region were retrieved by the scan.
- Resumed execution; purposefully let an enemy hit the player ship, lowering the number of lives to two. Next Scan: Value = 2. There was a considerable reduction in the candidate list.
- Repeated the procedure, allowing another hit (lives = 1), Next Scan: Value = 1, leaving 0x00A4F2BC as the only permanent address.
- Manually verified by changing the value at 0x001029AB to 5 in the memory editor; the HUD showed 5 lives right away, indicating that this is the right address.

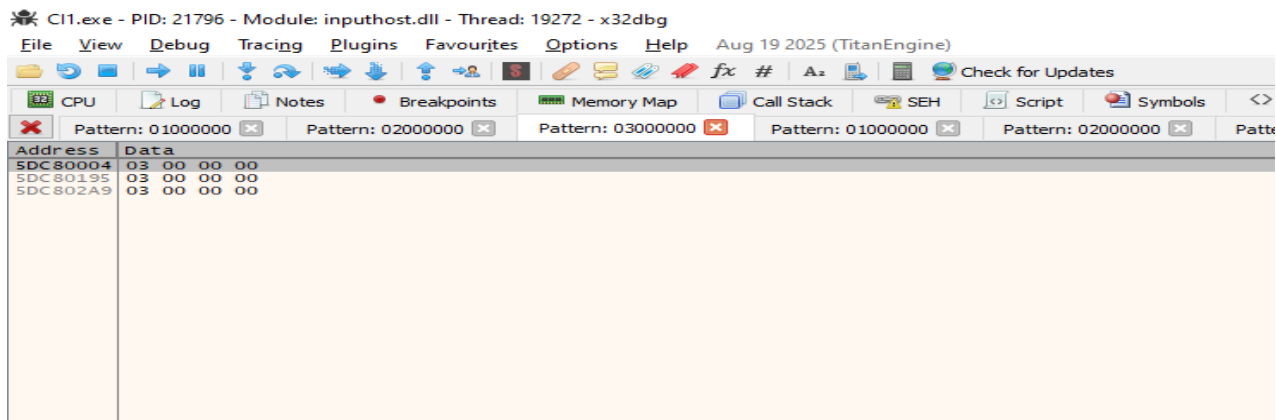


Figure 3 - Addresses which have value 3

3.2.2 Memory Write Breakpoint to Identify the Decrement Instruction

- A memory write breakpoint was set on address 0x001029AB in x32dbg.

Memory Map -> Right-click 0x001029AB -> Set Memory Breakpoint -> Write

- The game started over again. Execution stopped instantly when the player was hit by an enemy.
- Stepping back one instruction confirmed the preceding decrement:

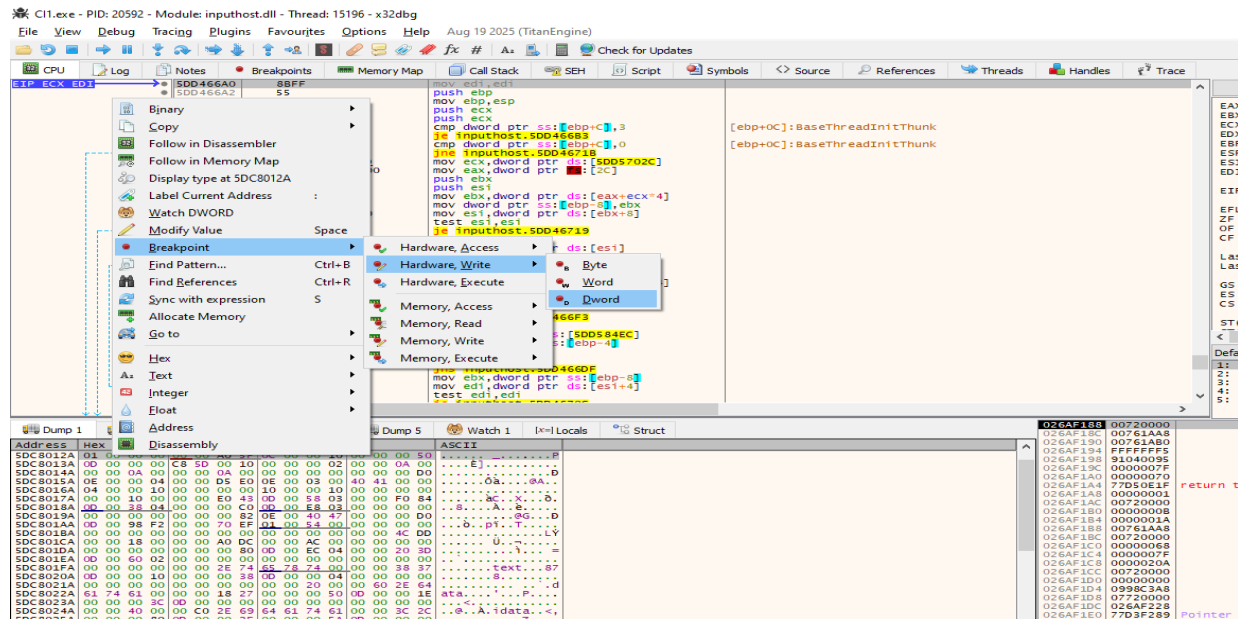


Figure 4 – set breakpoint for that address

4 Assembly-Level Modification

4.1 Identifying the Exact Instruction

Health decrement is caused by a 6-byte DEC instruction at 0x001029AB: FF 8F E4 00 00 00, which disassembles as DEC dword ptr ds:[edi+E4]. Every time the player sustains damage, this reduces the 32-bit health value kept at memory location EDI + 0xE4 by 1.

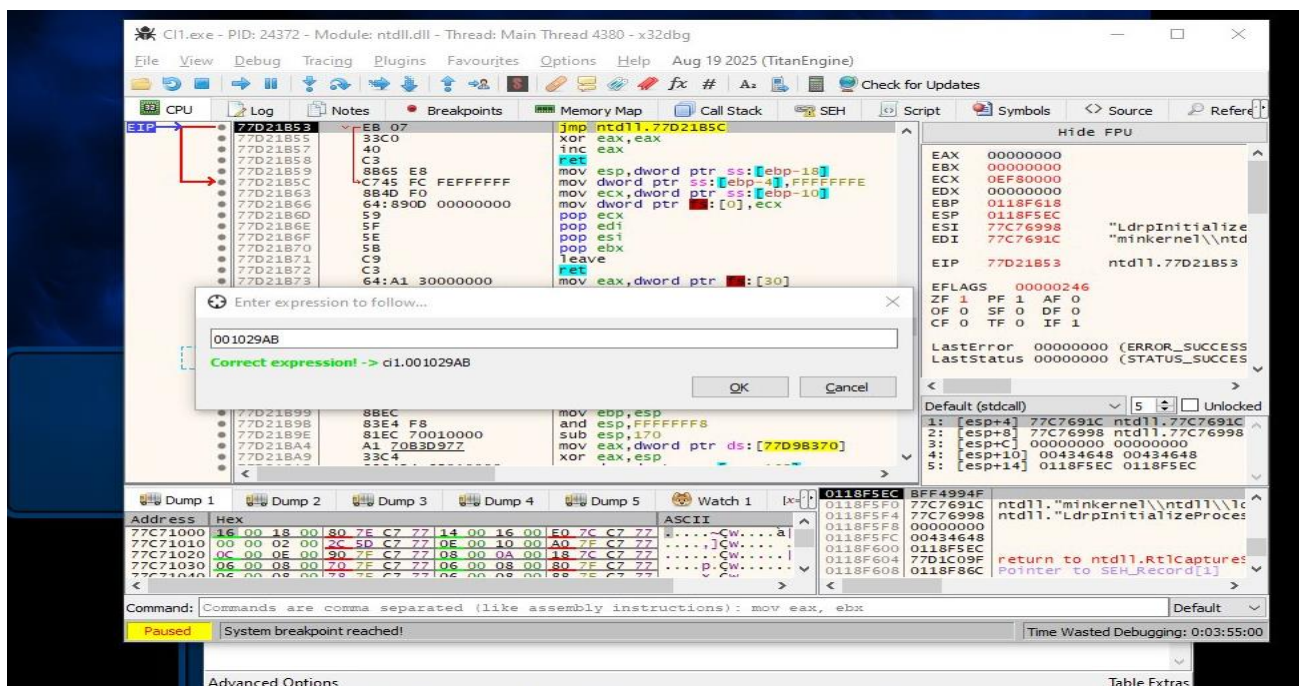


Figure 5 - 'Expression to Follow' dialog confirming target address 0x001029AB resolves to ci1.001029AB

4.2 Applying NOP Patches

Each byte of the 6-bit DEC instruction was changed to NOP (opcode 0x90 – No Operation) using x32dbg's Assemble dialogue (Space key on the instruction). All six bytes were replaced between addresses 0x001029AB through 0x001029AF. For every address, the assembler verified that "Instruction encoded successfully, Bytes: 90."

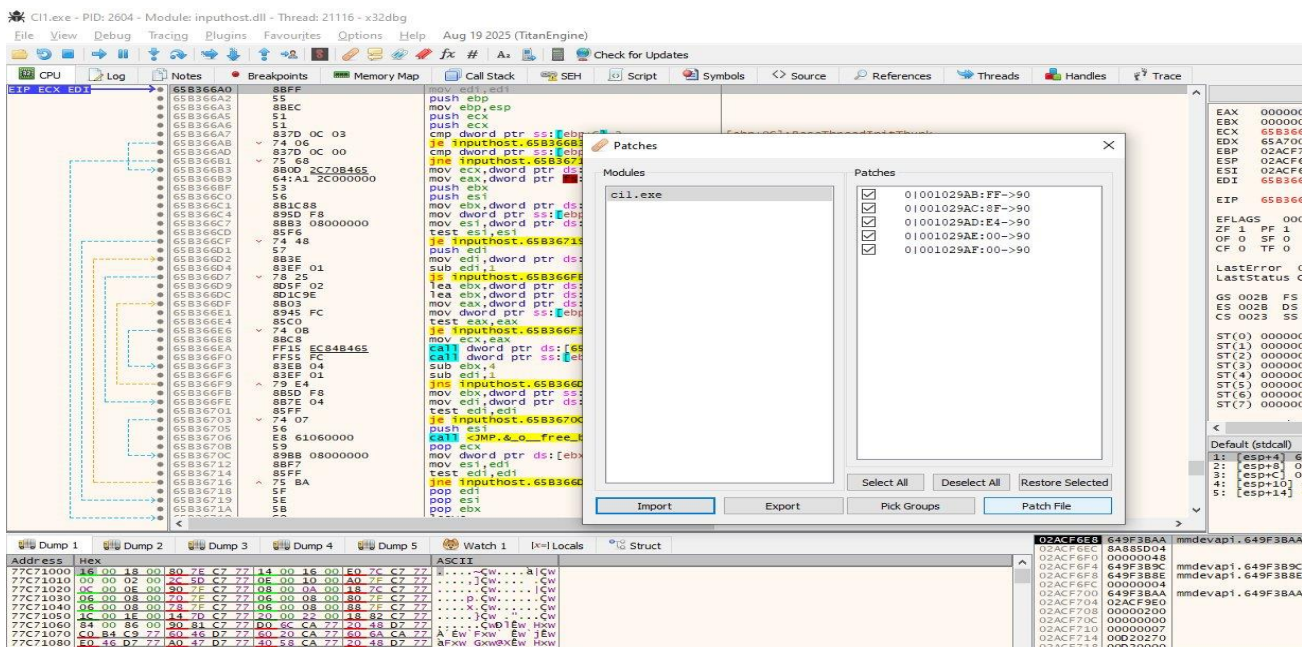
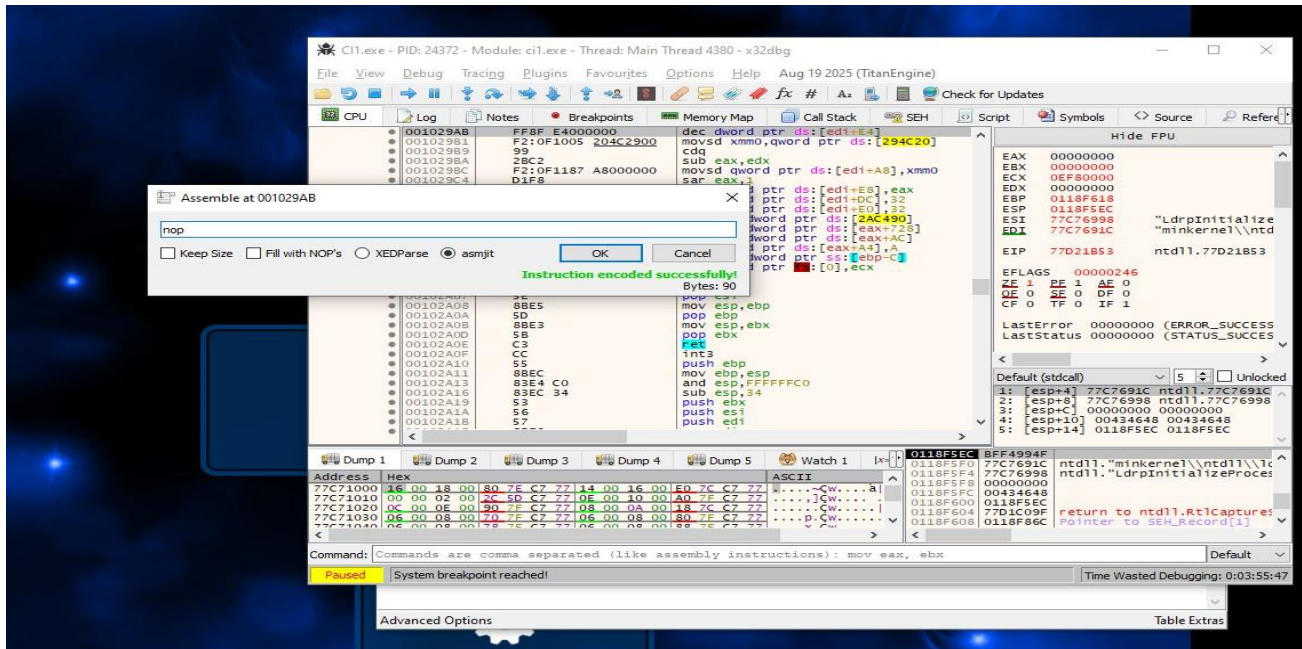


Figure 6 - Assemble dialog at 0x001029AB - 'nop' typed, 'Instruction encoded successfully, Bytes: 90' confirmed (DEC byte replaced)

4.3 Patch Verification

All five updated addresses (0x001029AB–0x001029AF) were displayed in the x32dbg Patches dialogue (File > Patch File), with 0x90 replacing the original bytes. Every patch entry was activated and verified. ChickenInvaders_patched1.exe is the name of the patched binary.

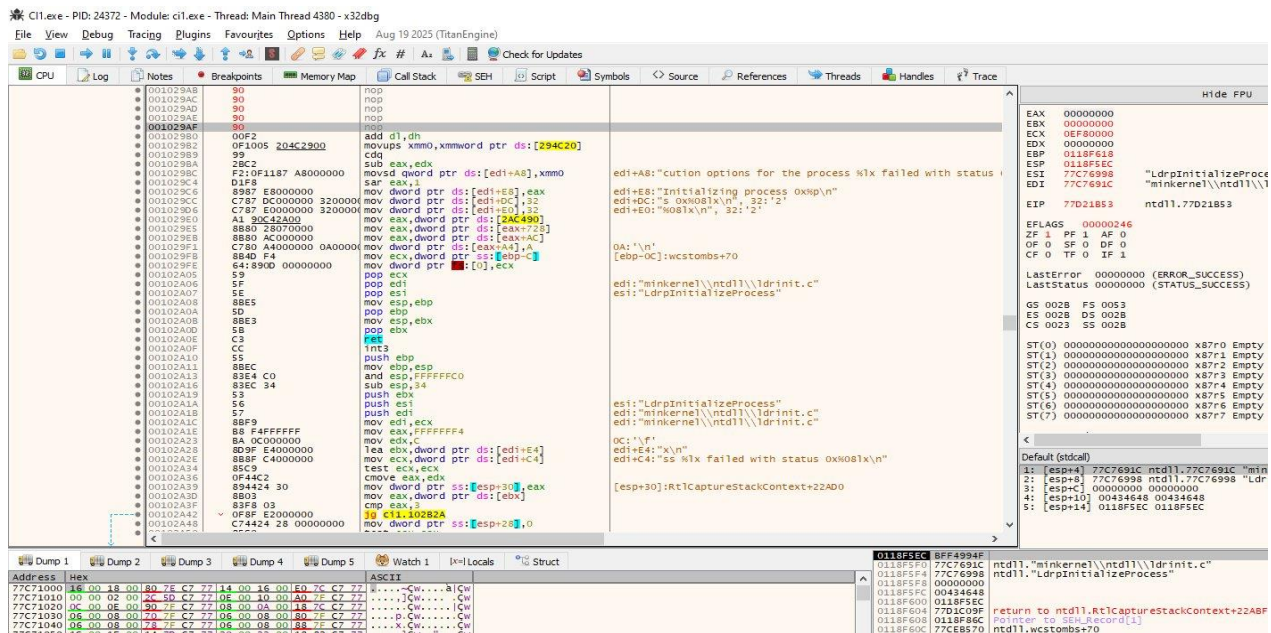


Figure 7 - Patches dialog — all 5 addresses (0x001029AB–0x001029AF) confirmed patched from original bytes to 0x90 (NOP), all checkboxes active

5 Security Analysis and Discussion

5.1 Why This Attack is Possible in Legacy Software

5.1.1 No Anti-Debugging Protection

The game does not use any anti-debugging techniques, such as timing assaults, IsDebuggerPresent() checks, SEH-based detection, or hardware breakpoint monitoring. x32dbg changed memory without interference, created breakpoints, and attached neatly.

5.1.2 Absence of Code Obfuscation or Packing

The binary for the game is completely unobfuscated. In x32dbg's disassembly view, the damage routine and its surrounding context may be easily read without the need for a deobfuscation step. Analyzing code obfuscation methods such as executable packing, control flow flattening, opaque predicates, and instruction substitution would take a lot more work. Without initially reverse-engineering the VM interpreter, instruction-level patching would be practically impossible due to virtual machine protection (VMProtect, Themida).

5.1.3 Game State in Unprotected Memory

In process virtual memory, the health value is kept as a simple 32-bit integer. There is no hardware-enforced access restriction, no checksum on the health value, and no memory encryption. It can be read or overwritten by any process that has enough OS rights.

5.1.4 No Runtime Integrity Verification

The game does not monitor its in-memory image while it is running, nor does it hash its code section at startup. It is completely undetectable to patch the DEC instruction to NOP, either on disc or in memory.

5.2 Missing Security Mechanisms Summary

Missing Security Mechanism	Consequence for This Attack
Anti-debugging APIs / checks	Debugger attached freely, all code and memory fully visible
Address Space Layout Randomization	Fixed instruction address 0x001029AB valid every session
Code obfuscation / VM protection	Damage routine directly readable and patchable in disassembly
Memory value encryption	Life counter stored as plain integer, trivially found by scan
Redundant value integrity checks	No detection when life counter is frozen or illegally modified
Code signing & hash verification	NOP patch accepted without any integrity check at load time

5.3 Mitigation Strategies for Modern Systems

5.3.1 Anti-Debugging Techniques

Use several detection layers, such as timing-based checks that identify execution pauses brought on by breakpoints, `IsDebuggerPresent`, `CheckRemoteDebuggerPresent`, `NtQueryInformationProcess` (`DebugPort`), and heap flag inspection. Layering greatly increases the cost of attacks, so no single strategy is adequate on its own.

5.3.2 Memory Value Obfuscation

Sensitive game variables should never be kept as simple numbers. Value splitting (health = $A + B$ stored at two different addresses), pointer chain indirection that modifies each frame, and XOR-masking with a rotating key are other methods. This makes the decrement of location obscure and defeats basic memory reading.

5.3.3 Runtime Integrity Verification

Compute a hash (CRC32 or SHA-256) over the process's own.text section on a regular basis, then compare it to a reference that has already been calculated. Any discrepancy, whether via code injection or NOP patching, results in resynchronization or termination. To avoid targeted bypass, distribute the bill among several locations.

5.3.4 Server-Side State Validation

Transfer the authoritative game state to a reliable server for online games. The server verifies every state change as the client renders and inputs. Periodic server check-ins can invalidate scores or achievements and indicate unusual sessions (e.g., zero health loss over a whole run) even in single-player games.

6 Conclusion

This assignment showed how to perform runtime manipulation and reverse engineering on Chicken Invaders 1 (2002). Static disassembly and dynamic register tracing were used to identify the player health decrement instruction (DEC dword ptr [edi+0xE4] at 0x001029AB, opcode FF 8F E4 00 00 00) using x32dbg. Effective invincibility was achieved by completely eliminating health loss by substituting NOP (0x90) instructions for all six bytes. ChickenInvaders_patched1.exe is the name of the patched binary.

A wide range of deficiencies were identified by security research, including the absence of ASLR, anti-debugging, code obfuscation, memory integrity verification, and code signing. When Chicken Invaders 1 was created, these omissions were commonplace, which reflects the low level of security knowledge in popular software development at the time. The suggested countermeasures, which include redundant integrity checking, memory value encryption, anti-debugging integration, ASLR, and VM-based obfuscation, are examples of current best practices that, taken as a whole, would make an equivalent attack on a modern application order of magnitude more challenging.

The exercise demonstrated how such manipulation is easily accomplished in legacy software due to the total lack of anti-debugging, code obfuscation, memory protection, and integrity checking. Anti-debugging layers, obfuscation, memory value encryption, runtime integrity checks, code signing, and server-side validation are examples of mitigation strategies that directly address these vulnerabilities. These techniques are the cornerstones of contemporary safe software architecture.

LINK - [SSS Game video](#)